

# Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



## 2028\_NeoColab\_Competitive Coding II\_L8

### VTU\_Week 5\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Arun is tasked with implementing a program to reverse a singly linked list. The program should populate the list elements by inserting them at the beginning, traverse the list and reverse the order of its elements.

Since Arun is new to programming, you have to guide him through the task.

#### ***Input Format***

The first line of input consists of an integer N, representing the linked list size.

The second line consists of N space-separated list elements.

#### ***Output Format***

The first line of output prints the list elements after inserting them at the beginning.

The second line prints the reversed linked list.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 5

23 85 47 62 31

Output: 31 62 47 85 23

23 85 47 62 31

### **Answer**

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node* insertBegin(struct Node* head, int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = head;  
    return newNode;  
}
```

```
void printList(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
struct Node* reverse(struct Node* head) {  
    struct Node *prev = NULL, *curr = head, *next;
```

```

while (curr != NULL) {
    next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}

return prev;
}

int main() {
    int n, x;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        head = insertBegin(head, x);
    }

    printList(head);

    head = reverse(head);

    printList(head);

    return 0;
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

In a computer science course project, students are required to write a program to insert values at the end of a singly linked list and remove the last node from the list. The program should traverse the list and delete the last node, adjusting pointers accordingly.

Assist students in the task.

### ***Input Format***

The first line of input consists of an integer N, representing the list size.

The second line consists of N space-separated integers, representing the list elements.

### ***Output Format***

The output prints space-separated integers, representing the list after the deletion of the last element.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 8

54 25 14 38 55 73 95 82

Output: 54 25 14 38 55 73 95

### ***Answer***

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node* insertEnd(struct Node* head, int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = NULL;
```

```
    if (head == NULL)  
        return newNode;
```

```
    struct Node* temp = head;
```

```
while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
return head;
}
```

```
struct Node* deleteLast(struct Node* head) {
    if (head == NULL || head->next == NULL)
        return NULL;
```

```
    struct Node* temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;

    return head;
}
```

```
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL)
            printf(" ");
        temp = temp->next;
    }
}
```

```
int main() {
    int n, x;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        head = insertEnd(head, x);
    }
}
```

```
    head = deleteLast(head);  
    printList(head);  
    return 0;  
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Dev is tasked with creating a program that efficiently finds the middle element of a linked list. The program should take user input to populate the linked list by inserting each element into the front of the list and then determining the middle element.

Assist Dev, as he needs to ensure that the middle element is accurately identified from the constructed singly linked list:

If it's an odd-length linked list, return the middle element. If it's an even-length linked list, return the second middle element of the two elements.

#### ***Input Format***

The first line of input consists of an integer  $n$ , representing the number of elements in the linked list.

The second line consists of  $n$  space-separated integers, representing the elements of the list.

#### ***Output Format***

The first line of output displays the linked list after inserting elements at the front.

The second line displays "Middle Element: " followed by the middle element of the linked list.

Refer to the sample output for formatting specifications.

### Sample Test Case

Input: 5

10 20 30 40 50

Output: 50 40 30 20 10

Middle Element: 30

### Answer

// You are using GCC

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node* insertFront(struct Node* head, int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = head;  
    return newNode;  
}
```

```
void printList(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
int findMiddle(struct Node* head) {  
    struct Node *slow = head, *fast = head;  
  
    while (fast != NULL && fast->next != NULL) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow->data;
```

```

}

int main() {
    int n, x;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        head = insertFront(head, x);
    }

    printList(head);

    int mid = findMiddle(head);
    printf("Middle Element: %d", mid);

    return 0;
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Saran is tasked with implementing a doubly linked list in data structures, allowing users to append elements to the end of the list, and remove the last element. He needs to ensure efficient memory management in the implementation.

Assist him with a suitable program.

##### **Input Format**

The first line of input consists of an integer n, representing the size of the list.

The second line consists of n space-separated integers, representing the elements of the list.

##### **Output Format**

The output displays integers, representing the list after removing the last

element, separated by a space.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 5

48 22 5 62 87

Output: 48 22 5 62

### **Answer**

// You are using GCC

#include <stdio.h>

#include <stdlib.h>

```
struct Node {  
    int data;  
    struct Node* prev;  
    struct Node* next;  
};
```

```
struct Node* insertEnd(struct Node* head, int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    newNode->prev = NULL;
```

```
    if (head == NULL)  
        return newNode;
```

```
    struct Node* temp = head;  
    while (temp->next != NULL)  
        temp = temp->next;
```

```
    temp->next = newNode;  
    newNode->prev = temp;
```

```
    return head;
```

```
}
```

```
struct Node* deleteLast(struct Node* head) {  
    if (head == NULL || head->next == NULL)  
        return NULL;
```

```
    struct Node* temp = head;  
    while (temp->next != NULL)  
        temp = temp->next;
```

```
    temp->prev->next = NULL;  
    free(temp);
```

```
    return head;
```

```
}
```

```
void printList(struct Node* head) {
```

```
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d", temp->data);  
        if (temp->next != NULL)  
            printf(" ");  
        temp = temp->next;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n, x;  
    scanf("%d", &n);
```

```
    struct Node* head = NULL;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &x);  
        head = insertEnd(head, x);  
    }
```

```
    head = deleteLast(head);
```

```
    printList(head);
```

```
    return 0;
```

```
}
```

**Status :** Correct

**Marks :** 10/10

### 5. Problem Statement:

Ravi is building a simple application to manage a doubly linked list of integers. He wants to insert new integers into the list and delete a node at a specific position. After deleting it, he needs to display the updated list.

Write a program to ensure that the task is done by performing insertion, deletion at a given position, and then printing the modified list.

#### **Input Format**

The first line contains an integer n, representing the number of elements to be inserted into the list.

The second line contains n space-separated integers, representing the values to insert.

The third line contains an integer pos, representing the position of the node to be deleted from the list.

#### **Output Format**

If the given position is valid, print "Node at position X with value Y deleted successfully".

If the position is invalid, print "Position X not found in the list."

In both cases, print

"Data present in the list:"

followed by the updated list elements on the next line, space separated."

Refer to the sample output for the formatting specifications.

**Sample Test Case**

Input: 5  
10 20 30 40 50  
4

Output: Node at position 4 with value 40 deleted successfully.  
Data present in the list:  
10 20 30 50

**Answer**

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *prev, *next;
};
```

```
struct Node* insertEnd(struct Node* head, int x) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = x; n->next = NULL; n->prev = NULL;
```

```
    if (!head) return n;
```

```
    struct Node* t = head;
    while (t->next) t = t->next;
```

```
    t->next = n;
    n->prev = t;
    return head;
}
```

```
int length(struct Node* h) {
    int c = 0;
    while (h) { c++; h = h->next; }
    return c;
}
```

```
struct Node* deletePos(struct Node* head, int pos, int *val, int *ok) {  
    int len = length(head);  
    if (pos < 1 || pos > len) { *ok = 0; return head; }
```

```
    *ok = 1;  
    struct Node* t = head;
```

```
    if (pos == 1) {  
        *val = t->data;  
        head = t->next;  
        if (head) head->prev = NULL;  
        free(t);  
        return head;  
    }
```

```
    for (int i = 1; i < pos; i++) t = t->next;
```

```
    *val = t->data;
```

```
    if (t->prev) t->prev->next = t->next;  
    if (t->next) t->next->prev = t->prev;
```

```
    free(t);  
    return head;  
}
```

```
void print(struct Node* h) {  
    while (h) {  
        printf("%d ", h->data);  
        h = h->next;  
    }  
}
```

```
int main() {  
    int n, x, pos, val, ok;  
    scanf("%d", &n);
```

```
    struct Node* head = NULL;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &x);  
        head = insertEnd(head, x);
```

```
}
```

```
scanf("%d", &pos);
```

```
head = deletePos(head, pos, &val, &ok);
```

```
if (ok)
```

```
    printf("Node at position %d with value %d deleted successfully.\n", pos, val);
```

```
else
```

```
    printf("Position %d not found in the list.\n", pos);
```

```
printf("Data present in the list:\n");
```

```
print(head);
```

```
return 0;
```

```
}
```

**Status :** Correct

**Marks :** 10/10